

Why 1D Quadratic Trend Filtering Beats the Current Graph Order-2 Fit

gflow graph trend filtering project notes

Build 2026-05-16 12:58:35 EDT

Abstract

This note explains the algorithmic difference between the benchmark winner `genlasso.trendfilter.ord2.cv` and the current `fit.graph.trend.filtering(order = 2)` implementation in `gflow`. The short answer is that they are not the same estimator, even on a one-dimensional path graph. The 1D trend filter penalizes third divided differences and has a three-dimensional unpenalized space corresponding to quadratic trends. The current graph order-2 implementation follows the Wang–Sharpnack–Smola–Tibshirani graph operator $D_w L_w$, which on a path has extra boundary rows and, in the current construction, only constants in its null space. This difference strongly explains why the path-graph benchmark still shows median RMSE 0.0322 for the best graph order-2 variant versus 0.0252 for the specialized 1D quadratic trend filter.

1 The Practical Question

In the curated Gaussian-mixture benchmark, the best method so far is

`genlasso.trendfilter.ord2.cv`,

with median RMSE to truth about 0.0252. The best graph trend-filtering variant currently tested is

`fit.graph.trend.filtering(order = 2, weight.rule = sqrt.conductance)`,

with median RMSE about 0.0322.

That gap is large enough that it deserves an algorithmic explanation. It is tempting to think that a path graph with order 2 graph trend filtering should reproduce ordinary 1D quadratic trend filtering. But our current graph operator does not do that. The two methods solve superficially similar ℓ_1 -penalized problems, but the penalty matrices encode different ideas about what should be considered “simple.”

2 What the 1D Quadratic Trend Filter Solves

Assume we observe ordered samples

$$x_1 < x_2 < \cdots < x_n, \quad y_i = f(x_i) + \varepsilon_i.$$

The 1D trend filter estimates fitted values

$$\theta = (\theta_1, \dots, \theta_n)^T, \quad \theta_i \approx f(x_i).$$

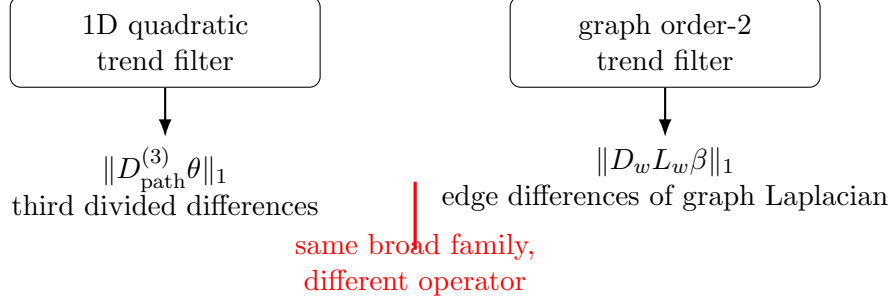


Figure 1: Both methods are ℓ_1 -adaptive smoothers, but they use different penalty operators. The penalty operator is the algorithmic heart of trend filtering.

For order $k = 2$, `genlasso::trendfilter(y, pos = x, ord = 2)` solves a generalized lasso problem of the form

$$\hat{\theta}_\lambda = \arg \min_{\theta \in \mathbb{R}^n} \left\{ \frac{1}{2} \|y - \theta\|_2^2 + \lambda \|D_{\text{pos}}^{(3)} \theta\|_1 \right\}.$$

The matrix $D_{\text{pos}}^{(3)}$ computes third-order divided differences. When the x_i are equally spaced, the interior stencil is

$$-\theta_i + 3\theta_{i+1} - 3\theta_{i+2} + \theta_{i+3}.$$

The sign does not matter because the penalty takes absolute values.

2.1 The Equal-Spaced Matrix

For $n = 7$, the equal-spaced order-2 trend-filtering penalty matrix is

$$D_{\text{path}}^{(3)} = \begin{bmatrix} -1 & 3 & -3 & 1 & 0 & 0 & 0 \\ 0 & -1 & 3 & -3 & 1 & 0 & 0 \\ 0 & 0 & -1 & 3 & -3 & 1 & 0 \\ 0 & 0 & 0 & -1 & 3 & -3 & 1 \end{bmatrix}.$$

There are $n - 3$ rows. This is important: the penalty does not try to define a third difference at the two boundaries. It only penalizes the interior third differences.

2.2 What This Makes Unpenalized

A third difference is zero for any quadratic sequence. Therefore

$$D_{\text{path}}^{(3)} \theta = 0$$

whenever θ_i lies on a quadratic curve in i , or more generally on a quadratic curve in x_i when divided differences are used. The null space has dimension 3:

$$\theta_i = a + bx_i + cx_i^2.$$

This is the sense in which order-2 trend filtering is “piecewise quadratic.” Where the third differences are zero, the fitted curve behaves like a quadratic. Where a third difference is nonzero, the method permits a knot, or local change in curvature.

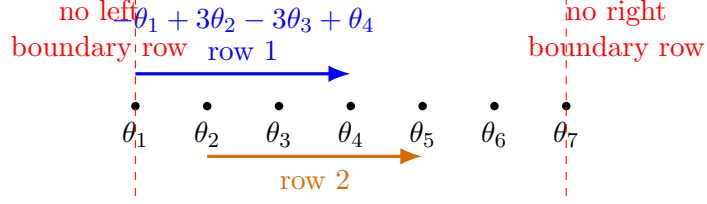


Figure 2: The 1D order-2 trend filter penalizes sliding four-point third differences. It has no boundary third-difference rows, which helps preserve a quadratic null space.

3 What the Current Graph Order-2 Fit Solves

The phase-2 gflow graph trend-filtering implementation follows the graph operators

$$D_w, \quad L_w = D_w^T D_w, \quad \Delta_w^{(3)} = D_w L_w.$$

For `order = 2`, the fit solves

$$\hat{\beta}_\lambda = \arg \min_{\beta \in \mathbb{R}^n} \left\{ \frac{1}{2} \|y - \beta\|_2^2 + \lambda \|D_w L_w \beta\|_1 \right\}.$$

Here D_w is a weighted oriented incidence matrix. For an edge $e = (i, j)$,

$$(D_w \beta)_e = \omega_e (\beta_j - \beta_i).$$

The weighted graph Laplacian is

$$L_w = D_w^T D_w.$$

Thus $L_w \beta$ is a graph-Laplacian signal on vertices, and $D_w L_w \beta$ computes edge differences of that Laplacian signal.

3.1 What Happens on an Unweighted Path

On an unweighted path with $n = 7$, the current graph order-2 penalty matrix is

$$DL = \begin{bmatrix} -2 & 3 & -1 & 0 & 0 & 0 & 0 \\ 1 & -3 & 3 & -1 & 0 & 0 & 0 \\ 0 & 1 & -3 & 3 & -1 & 0 & 0 \\ 0 & 0 & 1 & -3 & 3 & -1 & 0 \\ 0 & 0 & 0 & 1 & -3 & 3 & -1 \\ 0 & 0 & 0 & 0 & 1 & -3 & 2 \end{bmatrix}.$$

The middle rows look like third differences, up to sign. But the first and last rows are different:

$$-2\beta_1 + 3\beta_2 - \beta_3, \quad \beta_5 - 3\beta_6 + 2\beta_7.$$

These are boundary rows. They are not present in the ordinary 1D quadratic trend-filtering matrix.

Even more importantly, this matrix has $n - 1$ rows and rank $n - 1$ on a connected path. Its null space has dimension 1: constants only. It does not leave linear or quadratic trends unpenalized.

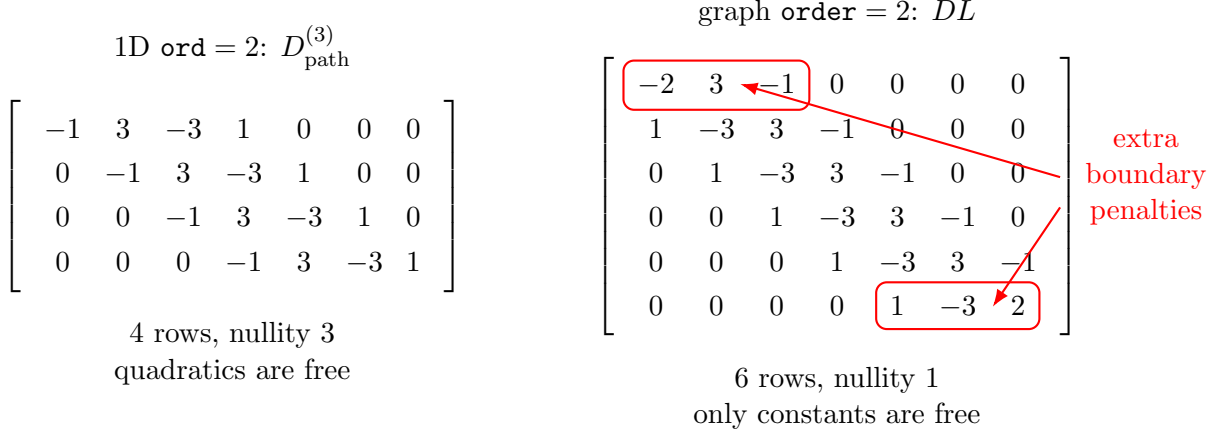


Figure 3: On a path, the current graph order-2 operator is not the same matrix as the ordinary 1D order-2 trend-filtering operator. The extra boundary rows and smaller null space change the estimator.

4 The Null Space Explains Much of the RMSE Gap

The null space is the set of fitted signals that receive no roughness penalty. It tells us what the smoother regards as globally simple.

Method	Penalty matrix	Unpenalized signals
1D ord = 2 trend filter	$D_{\text{path}}^{(3)}$	quadratic trends
current graph order = 2	$D_w L_w$	constants, for connected path

This matters for Gaussian mixtures. A Gaussian peak is smooth and curved. A piecewise quadratic estimator can approximate it with relatively few knots: many intervals can behave like quadratic arcs. The current graph order-2 operator does not grant the same freedom. It still penalizes linear and quadratic global behavior, especially through the boundary rows and through the graph-Laplacian construction.

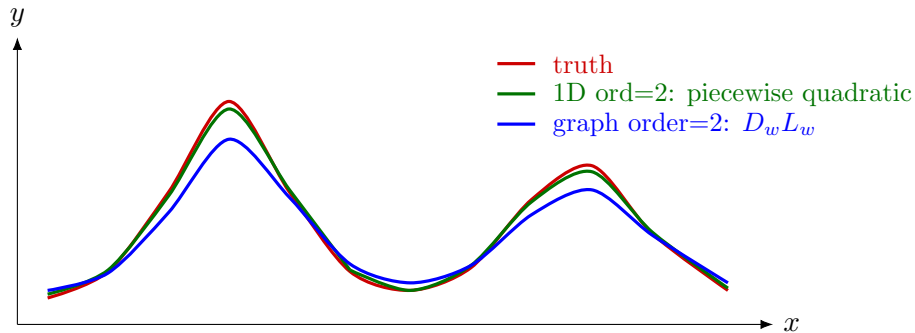


Figure 4: A Gaussian-mixture signal is smooth and curved. A 1D quadratic trend filter has exactly the right low-order local model: quadratic arcs separated by adaptive knots.

5 The Operators Also Handle Unequal Spacing Differently

The benchmark samples x_i randomly in $[0, 1]$, so the path spacing is not constant. The 1D trend filter uses `pos = x`. This means the penalty matrix is a divided-difference operator that accounts for unequal spacing:

$$D_{\text{pos}}^{(3)} = D^{(3)}(x_1, \dots, x_n).$$

Two gaps of different length are not treated as identical.

The current graph trend-filtering implementation receives only an adjacency list and edge weights. In the subset benchmark, the unit graph variant ignores spacing entirely, while the metric-weighted variants transform edge lengths into conductance-like weights. That is not the same as divided differences.

Divided differences approximate derivatives with respect to x . Conductance weights express how strongly adjacent vertices should be fused or smoothed. Both use edge lengths, but they encode different mathematical meanings.

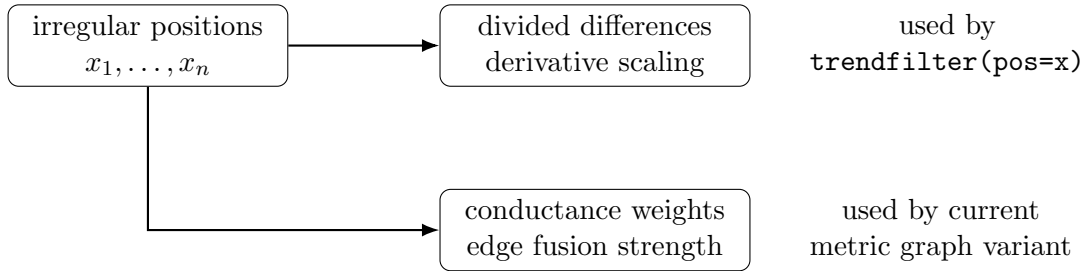


Figure 5: The specialized 1D trend filter and the graph metric-weighted variant both see the sample spacing, but they use it differently.

6 The Cross-Validation Procedures Differ Too

There is another important difference. The baseline uses

```
genlasso::cv.trendfilter(fit, mode = "lambda").
```

That routine is specialized for one-dimensional trend filtering. When it holds out interior points, it predicts them by interpolation from neighboring training positions. This matches the ordered 1D regression problem.

The current `fit.graph.trend.filtering()` cross-validation is graph generic. It forms a training selection matrix X , fits the generalized lasso to the observed vertices, and reads predictions for held-out vertices from the full coefficient vector. That is a reasonable generic graph CV strategy, but it is not the same criterion as `cv.trendfilter()`.

Thus even if the penalty matrix were made identical, the selected λ could still differ unless the CV procedure were also made identical.

7 Why the Benchmark Gap Is Expected

The gap

0.0322 versus 0.0252

is not mysterious once the operators are laid side by side.

1. The 1D baseline uses the exact order-2 path divided-difference operator.
2. That operator has a quadratic null space, which is well matched to smooth Gaussian peaks.
3. The current graph order-2 operator $D_w L_w$ does not have the same null space on a path; it leaves only constants unpenalized.
4. The current graph operator adds boundary penalties absent from the 1D trend-filtering matrix.
5. The graph metric-weighted variants use conductance weights, not the divided-difference scaling used by `trendfilter(pos=x)`.
6. The CV procedures select λ by different prediction rules.

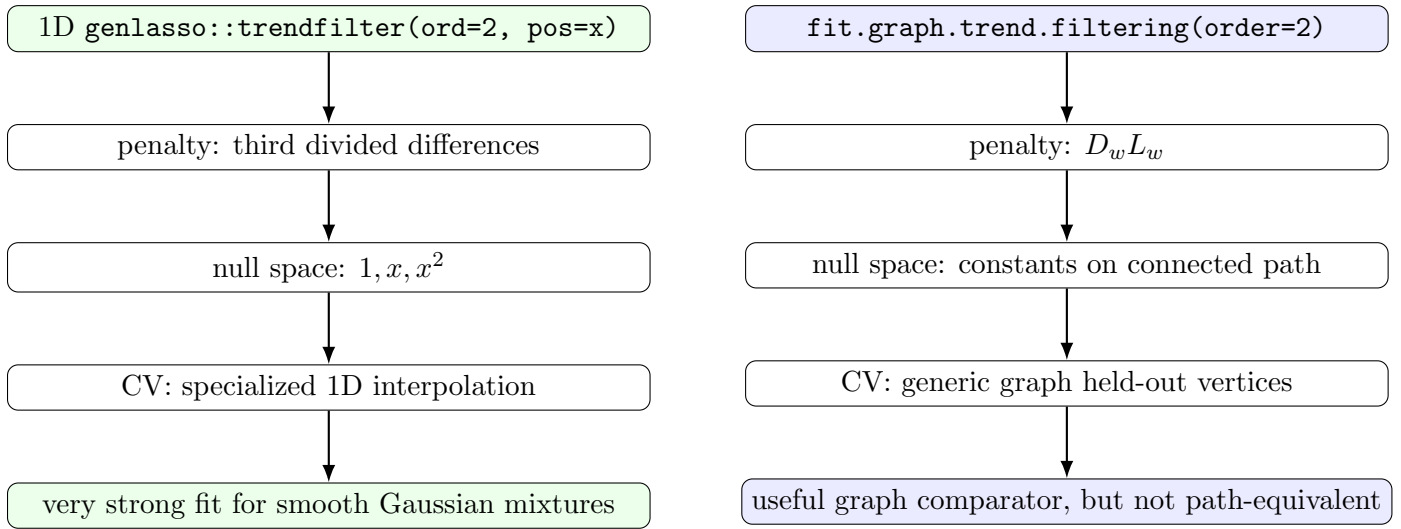


Figure 6: The two methods differ at several algorithmic layers: penalty, unpenalized space, spacing semantics, and cross-validation.

8 How to Make the 1D Case Match Exactly

If the goal is for `gflow` to reproduce the 1D trend-filtering algorithm on path graphs, then `fit.graph.trend.filtering(order = 2)` should not use $D_w L_w$ in that mode. It should use the same path divided-difference matrix as `genlasso::trendfilter()`.

8.1 Add a Path-Compatible Operator Family

One clean API direction is to separate the operator family from the order:

```
operator.family ∈ {"graph.laplacian", "path.divided.difference"}.
```

Then:

```
operator.family = "graph.laplacian" :    k = 0 : D_w,   k = 1 : L_w,   k = 2 : D_w L_w,
operator.family = "path.divided.difference" : k = 0 : D_pos^(1), k = 1 : D_pos^(2), k = 2 : D_pos^(3).
```

For a path graph, `pos` can be supplied directly, or reconstructed from metric edge lengths when the graph is a single path with two endpoints. For exact reproduction of `genlasso::trendfilter()`, the safest API is to require explicit vertex positions:

```
vertex.position = x.
```

8.2 Use the Same Solver and Lambda Selection

To reproduce the benchmark baseline exactly, matching the penalty matrix is not enough. The implementation should also call the same trend-filtering path and CV routines:

```
genlasso::trendfilter(y, pos = x, ord = 2)
```

and

```
genlasso::cv.trendfilter(..., mode = "lambda").
```

That suggests a second API switch:

```
lambda.selection = "path.cv"
```

or perhaps

```
solver = "genlasso.trendfilter"
```

for path-compatible fits. With the same y , the same x , the same operator, and the same CV rule, `gflow` should be able to match the 1D baseline's fitted values and RMSE up to numerical tolerance.

9 How to Keep the Graph Method Honest

The graph-Laplacian operator $D_w L_w$ is still scientifically meaningful. It is a real graph trend-filtering operator in the graph literature's sense. It asks for sparsity in edge differences of graph Laplacian values. That is a reasonable graph analogue when there is no global ordering and no notion of a polynomial in x .

But it should be presented as a graph method, not as a drop-in replacement for 1D trend filtering. In benchmark reports, the distinction should be explicit:

Question	Recommended operator
Do we want exact 1D trend-filtering behavior on a path?	Use path divided differences $D_{\text{pos}}^{(k+1)}$.
Do we want a graph-native Wang-style operator on an arbitrary graph?	Use incidence/Laplacian operators D_w , L_w , $D_w L_w$.
Do we want metric graph trend filtering?	Decide explicitly how metric lengths become ω_e , and do not assume this equals divided differences.

10 Recommended gflow Modifications

Here is the most direct path to make the 1D case match the benchmark winner.

1. Add a path-divided-difference operator constructor, possibly

```
path.trend.filtering.operator(x, order).
```

It should return the exact $D_{\text{pos}}^{(k+1)}$ matrix used by `genlasso`.

2. Add a mode to `fit.graph.trend.filtering()`:

```
operator.family = c("graph.laplacian", "path.divided.difference").
```

The default can remain graph-native for arbitrary graphs.

3. For `operator.family = "path.divided.difference"`, require `vertex.position` or reconstruct it only when the graph is provably a single path.
4. Add `lambda.selection = "path.cv"` or a solver option that uses `genlasso::cv.trendfilter()` instead of generic graph CV.
5. Add tests asserting exact agreement with `genlasso::trendfilter(y, pos = x, ord = order)` for `order = 0, 1, 2` on small path graphs.
6. Keep $D_w L_w$ as the graph-native order-2 operator, and document that it is not path-equivalent to ordinary quadratic trend filtering.

11 Summary

The RMSE difference 0.0322 versus 0.0252 is not mainly a tuning accident. The two estimators use different penalty matrices. The specialized 1D quadratic trend filter has the right null space and spacing semantics for smooth one-dimensional Gaussian-mixture regression. The current graph order-2 fit is a graph-native Laplacian-difference estimator; on a path, it is close in spirit but not algebraically equivalent.

To get the same algorithm and therefore the same 1D RMSE behavior, `gflow` should add a path-divided-difference operator family and, when exact parity is desired, reuse the same `genlasso::trendfilter()` and `cv.trendfilter()` machinery. That would let `gflow` offer both: an exact 1D trend-filtering compatibility mode for path graphs, and a graph-native trend-filtering mode for arbitrary weighted graphs.